

AI Usage DISCLOSURE FORM

1. Team Details

Team Name: The Cyclops

Project / Product Name: Fixby

Organization / Institution (if any): SRMIST-KTR

Submission Date: 30/09/2026

2. AI Usage Declaration

Did your team use any Artificial Intelligence (AI) in developing this project? Yes / No

Yes

3. Purpose of AI Usage (Brief Details)

Idea generation / brainstorming: Yes, used ChatGPT/Gemini to brainstorm the system architecture and caching logic.

Code generation or assistance: Yes, used AI coding assistants (e.g., Copilot, ChatGPT, Gemini) for boilerplate code, writing regular expressions, and debugging Python/Next.js components.

UI / UX design: Yes, utilized AI to brainstorm Tailwind CSS glassmorphism styles and frontend layouts.

Content creation: Yes, used LLMs for drafting prompt templates and generating sample user queries.

Data analysis: Yes, used AI/Embeddings for parsing unstructured SIIS telemetry constraints and mapping them to actionable device states.

Testing / debugging: Yes, used AI to generate Pytest cases and mock JSON responses.

Other: Core Engine Implementation (The product itself relies on LLMs and Vector Embeddings for runtime operation).

4. Feature Origin Classification (Please add/delete based on the number of the feature)

1. Feature Name: Structured Schema Extraction (Troubleshooting Plan Generation)

2. Self-Generated/AI-Generated/Both: Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification): AI tools/platform Used: Groq API (Qwen/GPT-oss variants) and Google Gemini (gemini-1.5-flash).

Prompt used: System prompt instructing the LLM to strictly output JSON matching the Samsung Goal/Action schema ("Follow these steps to perform this Troubleshooting", exactly 5-7 word action descriptions starting with "It will").

Output Summary: Generates the highly structured pure-JSON troubleshooting plan based on the user's vague query.

Modification: Self-generated Python code (validator.py) wraps the AI output to auto-repair any schema deviations, scrub hallucinated HTTP URLs, and enforce strict action categorization (moving critical actions to the end).

1. Feature Name: Semantic Deeplink Retrieval & Matching

2. Self-Generated/AI-Generated/Both: Both

3. Description (include AI tools/platform Used, prompt used , Output Summary, Modification): AI tools/platform Used: Sentence-Transformers (all-MiniLM-L6-v2) via HuggingFace/PyTorch.

Prompt used: N/A (Embedded direct text).

Output Summary: Translates the user's colloquial complaint into a dense mathematical vector to perform semantic cosine-similarity matching against the masked deeplinks.json catalog.

Modification: Self-generated Python code combines this AI embedding logic with domain-category keyword boosting (TF-IDF) and caching logic to ensure sub-300ms retrieval without full LLM inference overhead.

5. Ethical & Compliance Confirmation

AI usage complies with guidelines and policies: Yes

No proprietary or copyrighted data misused: I Agree

6. Declaration & Sign-Off

Name of Team Representative: Nishant Ranjan

Role: Team Lead

Signature: Nishant Ranjan

Date: 30/09/2026